

The Search for Meaning

LLMs through Linear Algebra

CS1807 Linear Algebra · Mini Project Study Guide
Avaneesh Rajesh Shetty · 1RUA25CSE0096

This guide covers every concept and equation from the mini project report — word embeddings, the self-attention mechanism, SVD and matrix compression, the linear-algebraic interpretation of hallucination, and the RAG experiment. Each equation is broken into its components with plain-language explanations.

Section	Topic
1	Word Embeddings & Semantic Geometry
2	Self-Attention Mechanism
3	SVD, Norms & Matrix Compression
4	Hallucination — A Null-Space Interpretation
5	RAG Experiment
6	All Equations — Quick Reference

Section 1 — Word Embeddings & Semantic Geometry

Words are symbols — they have no numeric meaning. To do mathematics with language, every word is mapped to a point in a high-dimensional real-number space. This mapping is the word embedding. The key property: geometrically close words share similar meaning.

Equation 1 — Embedding function definition

Word embedding definition

$$e : V \rightarrow \mathbb{R}^d, \text{ where } d \ll |V|$$

e	The embedding function. Maps each word to a unique vector (point in space).
V	The vocabulary — the complete set of all known words (may be 100,000+).
$\mathbb{R}^d$	d-dimensional real space. Every word becomes a list of d decimal numbers — its coordinates.
$d \ll V$	Compression: d might be 300-768 while V is 100,000. Far fewer dimensions than words.

Key insight: Why not $d = |V|$? That would be one-hot encoding — every word equidistant from every other. No geometry, no meaning.

Equation 2 — Semantic subspace

Semantic subspace

$$S_C = \text{span}\{ e(w) : w \in C \} \subseteq \mathbb{R}^d$$

$C \subseteq V$	A semantically coherent group — all animals, all royal titles, all verbs of motion.
$e(w)$	The embedding vector of word w — a single point in $\mathbb{R}^d$.
$\text{span}\{\dots\}$	All possible linear combinations of those vectors: $a_1 e(w_1) + a_2 e(w_2) + \dots$ This is a flat subspace of $\mathbb{R}^d$.

Key insight: Words in a semantic category do not scatter randomly — they lie in a low-dimensional subspace of the full embedding space.

Equation 3 — The famous word analogy

Vector analogy

$$e(\text{king}) - e(\text{man}) + e(\text{woman}) \approx e(\text{queen})$$

$e(\text{king}) - e(\text{man})$

Subtract the "male" direction from king. What remains is pure royalty — gender-neutralised.

$+ e(\text{woman})$

Add the "female" direction. You now have royalty + female = queen.

$\approx$

Approximate — not exact. On GloVe-300d, cosine similarity ≥ 0.92 . The subspaces are nearly but not perfectly orthogonal.

Key insight: This works because gender and royalty are approximately orthogonal subspaces — they do not interfere geometrically.

Equation 4 — Additive decomposition (the proof setup)

Additive decomposition

$$e(w) = e_{\text{royalty}}(w) + e_{\text{gender}}(w)$$

$e_{\text{royalty}}(w)$

The component in the "royalty" subspace. For man/woman, this is zero — no royalty meaning.

$e_{\text{gender}}(w)$

The component in the "gender" subspace. For king/queen these are opposite (male vs female).

Equation 5 — Full proof chain

Analogy proof

$$\begin{aligned} & e(\text{king}) - e(\text{man}) + e(\text{woman}) \\ &= [e_{\text{royalty}}(\text{king}) + e_{\text{gender}}(\text{man})] - e_{\text{gender}}(\text{man}) + e_{\text{gender}}(\text{woman}) \\ &= e_{\text{royalty}}(\text{king}) + e_{\text{gender}}(\text{woman}) \\ &= e_{\text{royalty}}(\text{queen}) + e_{\text{gender}}(\text{woman}) \\ &= e(\text{queen}) \quad \checkmark \end{aligned}$$

Line 1→2: expand. Line 2→3: gender(man) cancels. Line 3→4: king and queen share the same royalty vector. Line 4→5: collapse.

Loss function

$$L = \sum_{(w_t, w_c) \in D} \log p(w_c | w_t)$$

$$p(w_c | w_t) = e^{(v_c \cdot v_t)} / \sum_{w'} e^{(v_{w'} \cdot v_t)}$$

L

Total loss — sum over every (target, context) word pair seen in training. We maximise this.

 $\log p(w_c | w_t)$ "Given target word w_t , how likely is w_c to appear nearby?" Log turns products to sums — nicer gradients. **$e^{(v_c \cdot v_t)}$**

Numerator: exponentiated dot product of the two word vectors. High dot product = high alignment = predicted to co-occur.

 $\sum_{w'} e^{(v_{w'} \cdot v_t)}$

Denominator: sum over ALL vocabulary words. Normalises so probabilities sum to 1. This is the expensive part of training.

Key insight: Maximising L pushes co-occurring word pairs to have large dot products, moving them closer in R^d . The distributional hypothesis as an optimisation problem.

Section 2 — Self-Attention Mechanism

Static word embeddings give every occurrence of a word the same vector — "bank" in "river bank" and "bank transfer" are identical. Self-attention solves this by computing a contextualised representation for each token by attending to all other tokens, weighted by relevance.

Equation 7–10 — QKV linear projections

Linear transformations

$$W_Q \in \mathbb{R}^{(d \times d_k)}, W_K \in \mathbb{R}^{(d \times d_k)}, W_V \in \mathbb{R}^{(d \times d_v)}$$

$$Q = X W_Q \in \mathbb{R}^{(n \times d_k)} \leftarrow \text{"What am I looking for?"}$$

$$K = X W_K \in \mathbb{R}^{(n \times d_k)} \leftarrow \text{"What do I contain?"}$$

$$V = X W_V \in \mathbb{R}^{(n \times d_v)} \leftarrow \text{"What do I pass forward?"}$$

X $\in \mathbb{R}^{(n \times d)}$	Input: n tokens, each a d-dimensional embedding. One row per token.
W_Q, W_K, W_V	Learned weight matrices — the parameters trained during back-propagation.
Q (query)	"What am I looking for?" — each row q_i represents what token i needs from others.
K (key)	"What do I contain?" — each row k_j advertises what token j holds.
V (value)	"What do I actually pass forward?" — the content token j contributes if attended to.

Key insight: All three projections are plain matrix multiplications — no nonlinearity. The intelligence is entirely in the learned weights.

Equation 11 — Scaled dot-product attention (the full formula)

The complete attention formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) \cdot V$$

Q K^T	$n \times n$ matrix. Entry $(i,j) = q_i \cdot k_j = \text{dot product of query } i \text{ with key } j$. Measures geometric alignment.
/ $\sqrt{d_k}$	Scaling. For random unit vectors in $\mathbb{R}^{d_k}$, $\text{Var}(q \cdot k) = d_k$. Dividing by $\sqrt{d_k}$ normalises variance to 1, preventing softmax saturation.
softmax(...)	Projects $n \times n$ score matrix onto the probability simplex. Each row sums to 1. Gives attention weight w_{ij} for each token pair.
· V	Weighted sum of value vectors. Output for token $i = \sum_j w_{ij} \cdot v_j$ — a convex combination. This is the new contextualised representation.

Key insight: The output for each token is a weighted average of ALL other tokens' value vectors. Tokens that are geometrically aligned with the query receive more weight.

Equation 12 — Dot product written out explicitly

Inner product

$$[Q K^T]_{ij} = q_i \cdot k_j = \sum_{l=1}^{d_k} q_{il} * k_{jl}$$

$[Q K^T]_{ij}$ Entry (i,j) of the score matrix — the raw attention score from token i to token j.

$\sum q_{il} * k_{jl}$ Sum of elementwise products across all d_k dimensions — the Euclidean inner product written explicitly.

Key insight: Orthogonal vectors ($q \perp k$) give dot product zero — that token pair gets zero attention. Aligned vectors give maximum score.

Equation 13 — Cosine similarity

Cosine similarity

$$\cos \theta_{ij} = (q_i \cdot k_j) / (||q_i|| \cdot ||k_j||)$$

θ_{ij} The angle between query i and key j in R^{d_k} space.

$||q_i||, ||k_j||$ Vector magnitudes (lengths). Dividing by them means only direction matters, not length.

Range $\cos 0^\circ = 1$ (same direction), $\cos 90^\circ = 0$ (orthogonal), $\cos 180^\circ = -1$ (opposite). Always in $[-1, +1]$.

Key insight: When vectors are unit-normalised ($||q|| = ||k|| = 1$), the denominator is 1 and the dot product IS the cosine similarity.

Equation 14 — Softmax function

Probability simplex projection

$$\text{softmax}(z)_j = e^{(z_j)} / \sum_k e^{(z_k)}$$

$e^{(z_j)}$ Exponentiation: turns any real number into a positive number. Negative scores become small positives, not negative weights.

$\sum_k e^{(z_k)}$ Normalising constant. Dividing by it ensures all outputs sum to exactly 1.

Result $\in \Delta^{(n-1)}$ Output lives on the probability simplex — every entry in $[0,1]$, sum = 1. These become the attention weights w_{ij} .

Section 3 — SVD, Norms & Matrix Compression

Before neural embeddings, words were represented by co-occurrence matrices — huge, sparse, noisy. SVD extracts the meaningful signal with provably optimal compression. The Eckart-Young theorem guarantees no rank- k matrix can do better.

3.0 Co-occurrence matrix

Entry $A[i][j]$ counts how often word i appears within a context window of word j across the corpus. The window size is typically 5 words either side. The matrix is symmetric ($A = A^T$) since co-occurrence is mutual. For a 100,000-word vocabulary: 10 billion entries, extremely sparse.

Equation 15 — SVD existence theorem

The fundamental decomposition

$$A = U \Sigma V^T$$

$$U \in \mathbb{R}^{(m \times m)}, \text{ orthogonal } (U^T U = I)$$

$$\Sigma \in \mathbb{R}^{(m \times n)}, \text{ diagonal, } \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$$

$$V \in \mathbb{R}^{(n \times n)}, \text{ orthogonal } (V^T V = I)$$

U (left singular vectors)

Columns form an orthonormal basis for $\mathbb{R}^m$. They are the output-space directions that A maps to. Each column u_i is a unit vector perpendicular to all others.

Σ (singular values)

Diagonal matrix, entries non-negative, largest-to-smallest. σ_i measures how much A stretches in the i -th direction. σ_1 is maximum stretch.

V (right singular vectors)

Columns form an orthonormal basis for $\mathbb{R}^n$. They are the input directions A acts on. v_i maps to $\sigma_i u_i$ under A .

Orthogonal means

$U^T U = I$ and $V^T V = I$. Every column has length 1; different columns are perpendicular. These are pure rotations — no stretching, just reorientation.

Key insight: Geometric reading: $A = U \Sigma V^T$ means A rotates by V^T , then scales each axis by σ_i via Σ , then rotates by U . Every linear transformation decomposes this way.

Provably optimal compression

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$$

$$\|A - A_k\|_2 = \sigma_{k+1}$$

 $\sigma_i u_i v_i^T$

Each term is a rank-1 matrix (outer product of two unit vectors, scaled by σ_i). Summing k of these adds one direction of information per term.

 A_k

The rank- k approximation. Among ALL matrices of rank $\leq k$, this is closest to A . Not a heuristic — provably optimal under both norms.

 $\|A - A_k\|_F$

Frobenius error: square root of sum of discarded singular values squared. Small σ_i means discarding them barely increases error.

 $\|A - A_k\|_2$

Spectral error: simply σ_{k+1} , the largest discarded singular value. Measures worst-case residual stretch.

Key insight: In LSA, applying this to the co-occurrence matrix with $k = 300$ -500 preserves 95%+ of semantic structure at 200x compression.

Compression quality metric

$$\zeta_k = \left(\sum_{i=1}^k \sigma_i^2 \right) / \left(\sum_{i=1}^r \sigma_i^2 \right) \times 100\%$$

 $\sum_{i=1}^k \sigma_i^2$

Energy in the k kept components = $\|A_k\|_F^2$.

 $\sum_{i=1}^r \sigma_i^2$

Total energy in A = $\|A\|_F^2$.

 ζ_k

Fraction of variance retained. $\zeta_k = 0.95$ means 95% of information preserved. Typical: $k = 300$ achieves this for vocabulary-scale matrices.

3.1 Frobenius norm and spectral norm

Total matrix energy

$$\|A\|_F = \sqrt{\sum_i \sum_j a_{ij}^2} = \sqrt{\sigma_1^2 + \sigma_2^2 + \dots + \sigma_r^2}$$

 $\sum_i \sum_j a_{ij}^2$

Sum of squares of ALL entries. This is the Euclidean norm of A treated as a long vector.

 $= \sqrt{\sum \sigma_i^2}$

Equivalently: square root of sum of squared singular values. Proven via the trace identity $\|A\|_F^2 = \text{tr}(A^T A)$.

Maximum stretch

$$\|A\|_2 = \max_{\|x\|=1} \|Ax\| = \sigma_1$$

max $\|Ax\|$

Maximum length of the output when the input has length 1. The direction achieving this maximum is v_1 (the first right singular vector).

= σ_1

Equals the largest singular value. If you think of A as a transformation, the spectral norm is the worst-case amplification factor.

3.2 Principal axis theorem

For any real symmetric matrix S (like $A^T A$), the spectral theorem guarantees $S = Q \Lambda Q^T$ where Q is orthogonal and Λ is diagonal. The columns of Q are the eigenvectors (principal axes) and the diagonal of Λ holds the eigenvalues.

Principal axis theorem

$$A^T A = V \Sigma^2 V^T$$

Eigenvalues of $A^T A$: $\lambda_i = \sigma_i^2$

Eigenvectors of $A^T A$: columns of V (right singular vectors of A)

$A^T A$

Always symmetric and positive semidefinite — the spectral theorem applies.

$\lambda_i = \sigma_i^2$

The eigenvalues are the squared singular values. This is why SVD and PCA are so closely related.

Columns of V

Are the principal axes of the data ellipsoid — the directions of maximum variance in embedding space. SVD finds them and ranks them by importance.

Key insight: The first right singular vector v_1 points in the direction of maximum variance in the word embedding space. This is why SVD extracts the most meaningful dimensions first.

Section 4 — Hallucination: A Null-Space Interpretation

When an LLM confidently generates wrong information, the usual explanation is statistical. This project offers a linear algebra interpretation: if the informative part of a prompt's embedding lies in the null space of the model's transformations, that information is erased — and the model fills the gap with statistical priors.

Equation 18 — Rank-nullity theorem

Fundamental theorem of linear maps

$$\text{rank}(A) + \text{nullity}(A) = n$$
$$\text{ker}(A) = \{ x \in \mathbb{R}^n : Ax = 0 \}$$

n	Dimension of the input space — the number of columns in A.
rank(A)	Number of linearly independent output directions — dimension of the column space (image) of A.
nullity(A)	Dimension of $\text{ker}(A)$ — the null space. The number of input dimensions that A maps to zero.
Trade-off	$\text{rank} + \text{nullity} = n$ is a conservation law. High rank = small null space. Low rank (collapses many dimensions) = large null space. They must always sum to n.

Key insight: Example: A is 2x3 with rank 1. Then nullity = 3-1 = 2. Two full dimensions of input are completely destroyed by A.

Equation 19 — Hallucination hypothesis

The algebraic framing

$$A \cdot x_{\text{info}} \approx 0 \text{ when } x_{\text{info}} \in \text{ker}(A)$$

x_info	The informative component of the prompt's embedding — the part that carries actual query meaning.
ker(A)	The null space of the model's linear transformation. Any input here maps to zero — completely erased.
A · x_info ≈ 0	The informative signal is destroyed. Output is then driven by the model's statistical priors and biases — not your actual prompt content. This is hallucination.

Practical implications — prompt geometry

Prompt type	Geometric effect	Risk
Vague / broad	Embedding spread across many dimensions → higher null-space component	High

Specific, grounded	Embedding concentrated in high-variance directions → stays in column space	Low
RAG (retrieved docs)	Injects high-rank information directly → steers away from null space	Very low
Structured instructions	Reduces entropy of representation → less null-space contribution	Low

Section 5 — RAG Experiment

The project built a Retrieval-Augmented Generation (RAG) system using Llama 3.2 (3B parameters) running locally via Ollama, with nomic-embed-text for 768-dimensional embeddings. 15 math questions (calculus, linear algebra, statistics) were tested with and without retrieved context.

Condition	Correct	Score
Baseline (no RAG) — model relies only on its weights	10 / 15	66.7%
With RAG — top-3 facts retrieved and prepended	13 / 15	86.7%
Improvement	+3 questions	+20.0%

How RAG works — 4 steps, all linear algebra

Step 1 — Vectorise the knowledge base

Take 30 math facts. Run each through nomic-embed-text. Each document d_i becomes a 768-dimensional vector. Store all 30 as rows in $D \in \mathbb{R}^{(30 \times 768)}$.

Step 2 — Embed the query

The user's question is embedded by the same model into the same $\mathbb{R}^{768}$ space, giving query vector $q \in \mathbb{R}^{768}$. Same embedding space = comparable geometry.

Step 3 — Cosine similarity retrieval

Compute $\text{sim}(q, d_i)$ for every document. Select top-k by score. This is nearest-neighbour search in $\mathbb{R}^{768}$.

Step 4 — Augment and generate

Prepend retrieved facts to the prompt. Geometrically, this shifts the prompt's embedding into the high-information region of the model's column space — away from the null space.

Equation 21 — Cosine similarity for retrieval

Semantic search formula	
$\text{sim}(q, d_i) = (q \cdot d_i) / (q \cdot d_i)$	
q	Query vector — the user's question embedded into $\mathbb{R}^{768}$.
d_i	i -th document vector — one math fact, pre-embedded. One of 30 rows in $D \in \mathbb{R}^{(30 \times 768)}$.
$q \cdot d_i$	Dot product — numerator. Measures raw geometric alignment between query and document.
$ q \cdot d_i $	Magnitude product — denominator. Normalises so only direction (angle) matters, not length.
Range	Always in $[-1, +1]$. Score of 1 = identical direction, 0 = orthogonal, -1 = opposite. Higher = more semantically similar.

The reasoning gap — why 13.3% still failed

In 2 of 15 RAG cases, retrieval found the right context (similarity score > 0.85) but the model still answered incorrectly. The interpretation: even with the correct vector `x_info` present in the prompt, if the prompt structure places it near the null space of the attention heads, the model's "I don't know" priors outweigh the retrieved signal. This is empirical evidence for the hallucination hypothesis from Section 4.

Section 6 — All Equations: Quick Reference

Eq	Formula	What it does
1	$e : V \rightarrow \mathbb{R}^d, d \ll V $	Maps words to d-dimensional vectors
2	$S_C = \text{span}\{e(w) : w \in C\}$	Defines a semantic subspace for word cluster C
3	$e(\text{king}) - e(\text{man}) + e(\text{woman}) \approx e(\text{queen})$	Word analogy via vector arithmetic
4	$e(w) = e_{\text{royalty}}(w) + e_{\text{gender}}(w)$	Additive decomposition of embedding into subspaces
5	Full proof chain (5 steps)	Proves analogy holds when subspaces are orthogonal
6	$L = \sum \log p(w_c w_t); p = e^{\langle v_c, v_t \rangle} / \sum e^{\langle v_c, v_l \rangle}$	Skip-gram Ls: maximise co-occurrence dot products
7-10	$Q=XW_Q, K=XW_K, V=XW_V$	Three linear projections of input into QKV subspaces
11	$\text{Attn}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$	Scaled dot-product attention — full formula
12	$[QK^T]_{ij} = q_i \cdot k_j = \sum_l q_{il} k_{jl}$	Dot product of query i with key j (inner product)
13	$\cos \theta_{ij} = (q_i \cdot k_j) / (q_i \cdot k_j)$	Cosine similarity between query and key vectors
14	$\text{softmax}(z)_j = e^{z_j} / \sum_k e^{z_k}$	Projects scores to probability simplex (sum = 1)
15	$A = U \Sigma V^T$	SVD: any matrix = rotation × scale × rotation
16	$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T; \text{error} = \sqrt{\sum_{i=k+1}^n \sigma_i^2}$	Best rank-k approximation (Eckart-Young theorem)
17	$\zeta_k = (\sum_{i=1}^k \sigma_i^2) / (\sum_{i=1}^n \sigma_i^2)$	Fraction of variance retained by rank-k SVD
F	$ A _F = \sqrt{\sum_{ij} a_{ij}^2} = \sqrt{\sum \sigma_i^2}$	Frobenius norm: total matrix energy
S	$ A _2 = \max_{ x =1} Ax = \sigma_1$	Spectral norm: maximum stretch = largest singular value
PA	$A^T A = V \Sigma^2 V^T$	Right singular vectors = eigenvectors of $A^T A$
18	$\text{rank}(A) + \text{nullity}(A) = n; \ker(A) = \{x: Ax=0\}$	Rank-nullity: conservation law for linear maps
19	$A \cdot x_{\text{info}} \approx 0 \text{ when } x_{\text{info}} \in \ker(A)$	Hallucination hypothesis: null-space erases prompt signal
21	$\text{sim}(q, d_i) = (q \cdot d_i) / (q \cdot d_i)$	Cosine similarity used for RAG retrieval

Key insight: Presentation tip: The three most likely deep-dive questions are (1) why $\sqrt{d_k}$ in the attention formula, (2) the king-man+woman proof, and (3) how the Eckart-Young theorem guarantees optimality. Make sure you can explain all three from first principles without referring to the formula sheet.